

Workshop on Advanced Sampling Methodologies

AI and ML for sample selection

Dr. Gianluca Boo, WorldPop, University of Southampton

2025-09-23

Workshop's agenda

- **Day 1:** Introduction to QGIS
- **Day 2:** Automatic update of census EA and national sampling frame
- **Day 3:** Introduction to R programming
- **Day 4:** Introduction to sampling design
- **Day 5:** Advanced R programming
- **Day 6:** Earth observation (EO) for sampling design
- **Day 7: AI and machine learning for sample selection**
- **Day 8:** Simulation and optimization of sampling strategies
- **Day 9:** R Shiny for sampling design evaluation
- **Day 10:** Wrap-up and workshop evaluation

Course materials

1. Today's course materials are on **Github**:
https://github.com/gianlucaboo/sampling_design_workshop
2. Click on **Day 7: AI and machine learning for sample selection**
3. **Copy the url** and **paste it** on <https://download-directory.github.io>
4. Save the zipped directory on the workshop and unzip it

Today's agenda

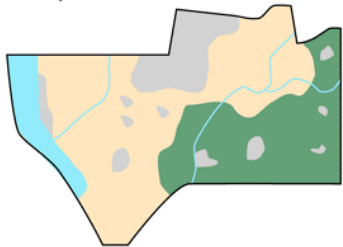
- Introduction to AI and ML in sampling
- K-means clustering for stratification
- Random forest for sample weighting and prediction
- Implementation in R with tidymodels
- Practical exercises

AI and ML in sampling

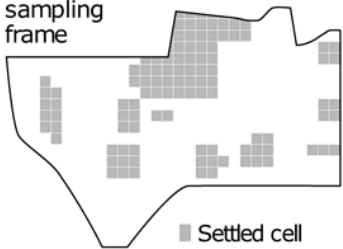
- Improve **sampling efficiency** and representativeness
- Detect patterns in **complex population data**
- **Reduce bias** and **improve stratification**
- **Reduce sample** size while maintaining accuracy

A Sampling frame

A1 Study area

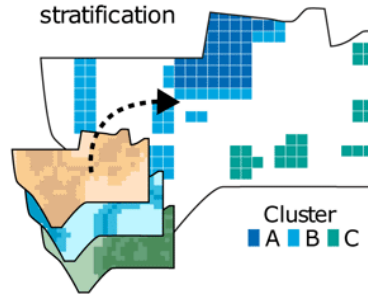


A2 Gridded sampling frame

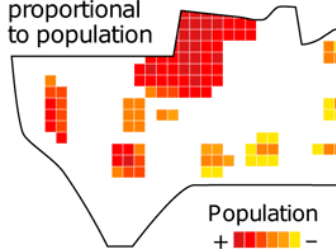


B Sampling design

B1 Contextual stratification

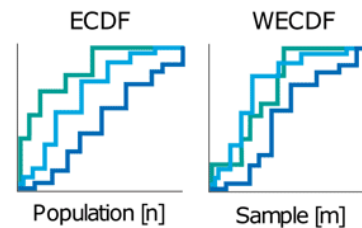


B2 Probability proportional to population

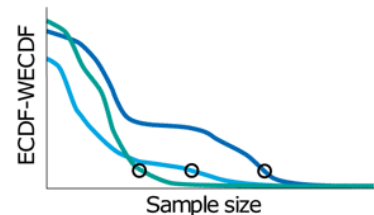


C Estimator

C1 Design evaluation



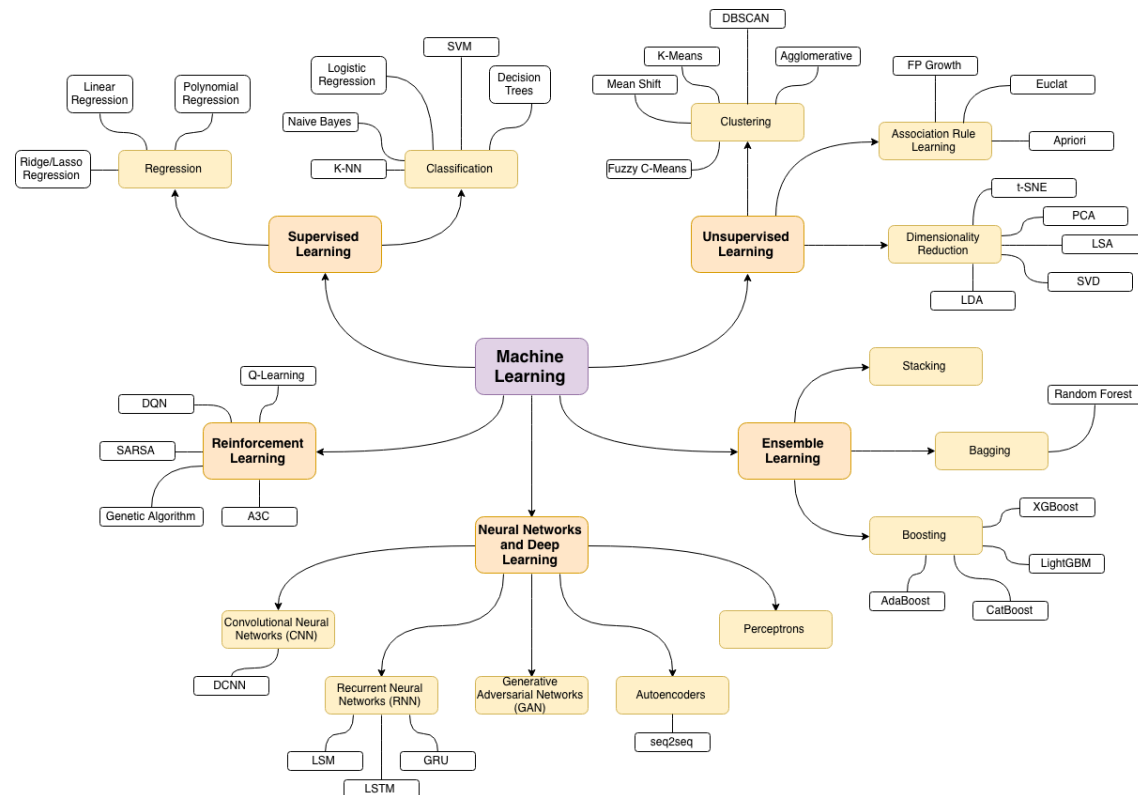
C2 Sample size estimation



Overview of AI/ML

There are many AI/ML algorithms, where the most common are:

- **Supervised learning:** prediction based on known outcomes
- **Unsupervised learning:** finding patterns/clusters without labeled outcomes



Some ML algorithms (source: Vas3K.blog).

ML workflow

ML has a **training**, **validation**, and **prediction** workflow.

edureka!

A ML workflow (source: Edureka!).

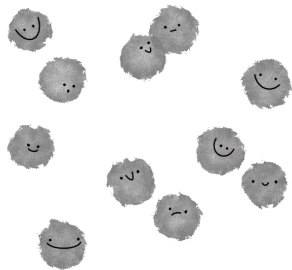
K-means clustering

- This is an **unsupervised learning** method
- Detect **natural groups** in population data
- Useful for creating strata for **representative samples**
- **k-means**: widely used for **stratification**
- Partition population into **K clusters**
- Minimize **within-cluster variance**

k-means
clustering

- assign each observation to one of **k** clusters based on the nearest cluster centroid.

OBSERVATIONS



cluster
CENTROIDS



@allison-horst

K-means in R

A few key facts about the **Iris data set**:

- Presented by to British statistician and biologist **Ronald Fisher** in his 1936 paper.
- The data set consists of **50 samples** from each of **three species** of Iris.
- Four features were measured: the **length and the width of the sepals and petals**.



Iris setosa (source: Wikipedia)



Iris versicolor (source: Wikipedia)



Iris Virginica (source: Wikipedia)

K-means in R

A first glimpse at **the data set structure**:

```
1 library(tidyverse)
2
3 # Explore dataset
4 iris |>
5   head(n=2)
6
7 # Plot species
8 ggplot(iris, aes(x=Sepal.Length, y=Sepal.Width, color=factor(Species))) +
9   geom_point(size=3) +
10  theme_minimal() +
11  labs(color="Cluster")
```

 Try to run the code in your Script Editor.

K-means in R

Running a K-means clustering with **k=3**:

```
1 library(tidymodels); library(tidyverse); library(cluster)
2
3 ## Remove species
4 data <-
5   iris |>
6   select(-Species)
7
8 ## K-means clustering
9 kmeans_fit <-
10   kmeans(scale(data), centers = 3) # it is important to scale and center the data
11
12 ## Attach clusters to dataset
13 data <-
14   data |>
15   mutate(Cluster=kmeans_fit$cluster,
16           Species=iris |> pull(Species))
17
18 ## Plot the results
```

 Try to run the code in your Script Editor. Try to use 4 or 5 clusters.

K-means in R

The output of `kmeans_fit` is a **list of vectors**:

- `cluster` contains information about each point
- `centers`, `withinss`, and `size` (3 values) contain information about each cluster
- `totss`, `tot.withinss`, `betweenss`, and `iter` (1 value) contain information about the full clustering

```
1 library(purrr); library(tidymodels); library(tidyverse); library(cluster)
2
3 k_means_multiple_fit <-
4   c(1:9) |>
5   map (function(k) {
6     kmeans(scale(data |> select(-c(Cluster, Species))), centers = k) |>
7       _$cluster |>
8       data.frame()
9   }) |>
10  list_cbind() |>
11  cbind(iris) |>
12  pivot_longer(cols=starts_with("kmeans"), names_to="k", values_to="Cluster")
13  mutate(k=k |> str_sub(-1, -1))
14
15 ggplot(k_means_multiple_fit, aes(x=Sepal.Length, y=Sepal.Width, color=factor(k)))
```

```
16 geom_point(size=2) +  
17 theme_minimal() +
```

 Try to run the code in your Script Editor.

K-means in R

The `tot.withinss` column (total within sum of squares) represents the variance within the clusters. It generally decreases as `k` increases, and a bend (or “elbow”) indicates that additional clusters beyond the third have little value.

```
1 library(purrr); library(tidymodels); library(tidyverse); library(cluster)
2
3 ## Run the clustering algorithm for different k
4 k_means_multiple_withinss <-
5   c(1:9) |>
6   map (function(k) {
7     kmeans(scale(data), centers = k) |>
8       _$tot.withinss |>
9       data.frame()
10  }) |>
11  list_cbind() |>
12  pivot_longer(cols=everything(), names_to="k", values_to="tot.withinss") |
13  mutate(k=k |> str_sub(-1, -1) |> as.numeric())
14
15 ## Plot the results
16 ggplot(k_means_multiple_withinss, aes(x=k, y=tot.withinss)) +
17   geom_line()+
```

10 `geom_point(aes(size = 100))`

 Try to run the code in your Script Editor.

Random forest

Decision trees

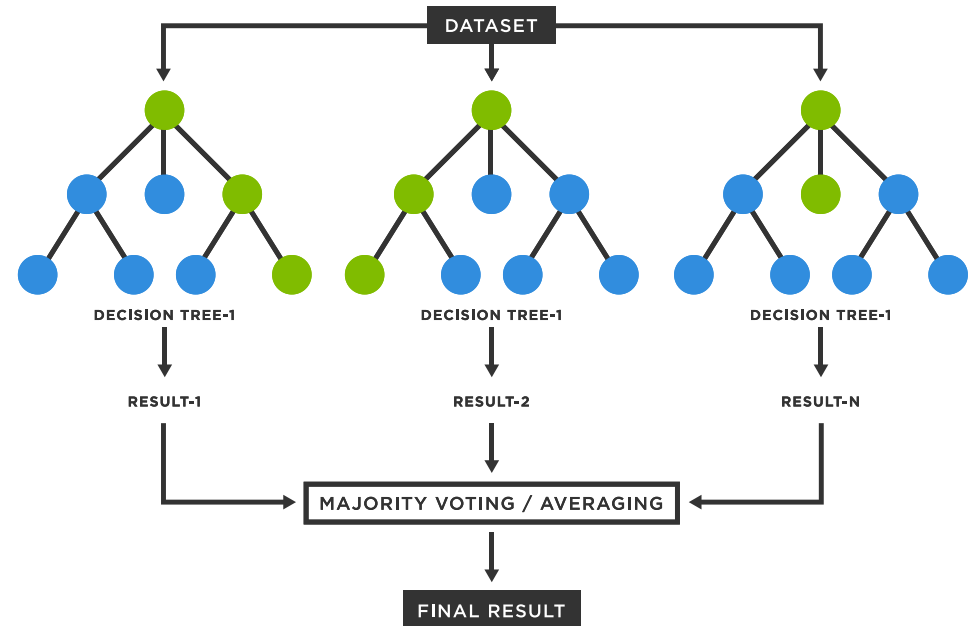
- A decision tree splits data into branches based on feature values (e.g., "Is age > 30?").
- The algorithm randomly selects samples (with replacement) from the training data to build each tree.

Random feature selection

- At each split in a tree, it considers only a random subset of features (not all of them).
- This makes the trees more diverse and reduces correlation among them.

Voting or averaging

- For **classification**, each tree votes for a class; the majority wins.
- For **regression**, the forest prediction is the average of all tree outputs.



Random forest diagram (source: spotfire.com).

Random forest in R

Creating a **random forest model** to predict **Petal.Length**

```
1 library(tidymodels) # Load the tidymodels package for machine learning wor
2
3 # Split data
4 set.seed(123) # Set a random seed for reproducibility
5 data_split <- initial_split(iris |> select(-Species), prop = 0.8)
6 # Randomly split the iris dataset (without Species column) into training (8
7
8 train <- training(data_split) # Extract the training portion of the split
9 test <- testing(data_split)   # Extract the testing portion of the split
10
11 # Define recipe and model
12 rf_recipe <- recipe(Petal.Length ~ ., data = train)
13 # Define a preprocessing recipe where Petal.Length is the outcome
14 # and all other variables are predictors
15
16 rf_model <- rand_forest(trees = 500) |> # Specify a random forest model
17   set_engine("ranger", importance = "permutation") |>
18   # Use the 'ranger' engine and compute variable importance with permutatio
```

 Try to run the code in your Script Editor.

Random forest in R

The model is **trained on a subset of the data**

```
1 # Workflow
2 rf_wf <-
3   workflow() |>           # Initialize an empty workflow
4   add_recipe(rf_recipe) |> # Add the preprocessing recipe (defines pre
5   add_model(rf_model)     # Add the random forest model specification
6
7 # Fit model
8 rf_fit <-
9   rf_wf |>               # Take the workflow (recipe + model)
10  fit(data = train)      # Train (fit) it on the training dataset
11
12 # Predict
13 rf_preds <-
14   predict(rf_fit, test) |> # Generate predictions on the test dataset
15   bind_cols(test)         # Combine predictions with the original test
```

 Try to run the code in your Script Editor.

Random forest in R

A regression model can be **evaluated by comparing its predicted values to the true observed values**. Common metrics include:

- **MAE**: mean absolute error (lower is better)
- **RMSE**: root mean squared error (lower is better)
- **R²**: proportion of variance explained (higher is better)

```
1 rf_preds |>  
2   metrics(truth = Petal.Length, estimate = .pred)
```

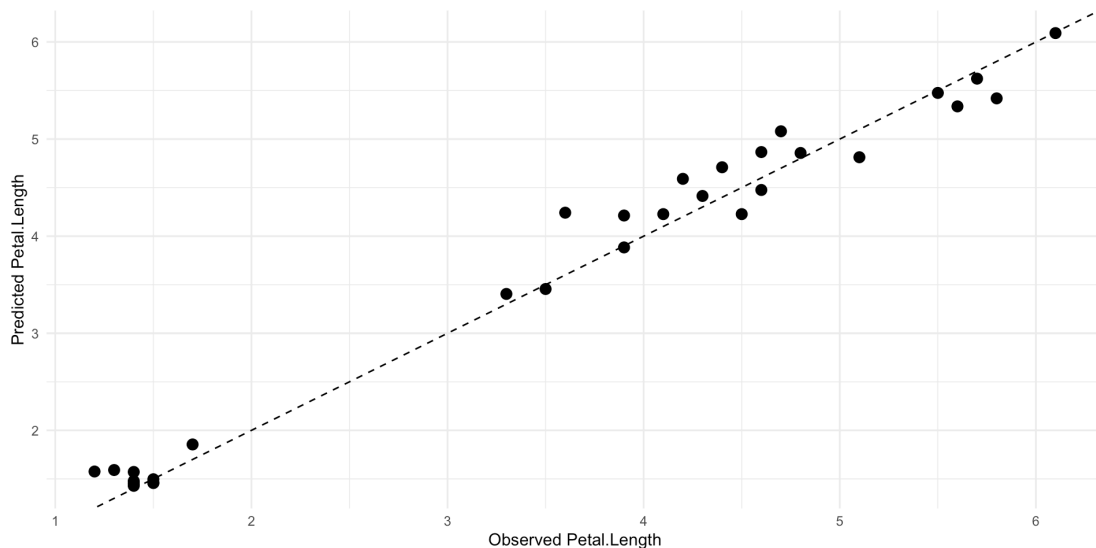
```
# A tibble: 3 × 3  
  .metric .estimator .estimate  
  <chr>    <chr>         <dbl>  
1 rmse     standard        0.238  
2 rsq      standard        0.981  
3 mae      standard        0.181
```

 Try to run the code in your Script Editor.

Random forest in R

Arguably, the best way to **explore model performance** is to plot observed versus predicted values.

```
1  ggplot(data=rf_preds, aes(x = Petal.Length, y = .pred)) +  
2  geom_point(size=3, color="black") +  
3  geom_abline(slope=1, intercept=0, linetype="dashed") +  
4  theme_minimal() +  
5  labs(x="Observed Petal.Length", y="Predicted Petal.Length")
```



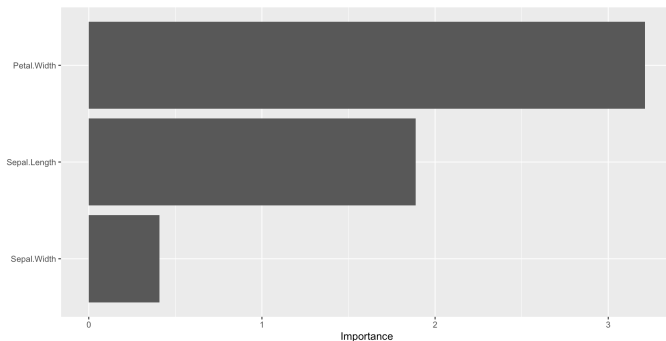
Try to run the code in your Script Editor.

Random forest in R

Variable importance (VIP) helps us understand which predictors the model relied on most when making predictions.

By plotting the top features, we can see **which variables contribute most** to predicting the outcome—in this case, **Petal.Length**.

```
1 library(vip)
2
3 rf_fit |>
4   extract_fit_parsnip() %>%
5   vip::vip(num_features = 3)
```



 Try to run the code in your Script Editor.

Random forest applications and tutorials

WorldPop develops random forest models to disaggregate census data at high spatial resolution globally. Tutorials are available at [**https://www.worldpop.org/tutorials/**](https://www.worldpop.org/tutorials/).

Using ML for sampling

K-means clustering

- Creates **data-driven strata** by grouping similar units
- Improves **representativeness** when no prior strata exist
- Reduces **intra-stratum variance**, leading to smaller required sample sizes

Random forest

- Predicts **key population characteristics** (e.g., poverty, health indicators)
- Generates **sample weights** or **inclusion probabilities** from covariates
- Captures **non-linear relationships** and interactions often missed by classical models

Combined approach

- Use **random-forest predictions** as inputs to **k-means** to create strata based on both observed and modeled variables
- Enables **adaptive, evidence-based sampling frames** for large-scale surveys

Practical Considerations

ML is powerful but should be **handled carefully**. Below a few key parameters to consider:

- Choosing **K** in k-means (elbow method)
- **Number of trees** in random forest
- Handle **missing data** appropriately
- **Scale/normalize** features before clustering

 Please do consult additional references (at the end of this presentation).

More models in `tidymodels`

tidymodels is a collection of R packages for modeling and machine learning.

Built on the **tidyverse** philosophy: consistent, tidy syntax.

Provides tools for the entire modeling workflow:

- **rsample** for data splitting, resampling
- **recipes** for preprocessing and feature engineering
- **parsnip** for model specification
- **tune** for hyperparameter tuning
- **yardstick** for model evaluation

Linear models

- `linear_reg()`: linear regression
- `logistic_reg()`: logistic regression

Tree-based models

- `decision_tree()`: classification/regression trees
- `rand_forest()`: random forests
- `boost_tree()`: gradient boosting

Support vector machines

- `svm_linear()`, `svm_poly()`, `svm_rbf()`: SVM variants

Nearest neighbors

- `nearest_neighbor()`: k-nearest neighbors (KNN)

Generalized additive models

- `gen_additive_mod()`: flexible smooth models

Neural networks

- `mlp()`: multi-layer perceptron models

 Many more available through **extensions** (e.g., `xgboost`, `keras`, `glmnet`).

tidymodels workflow

Below the key components of a `tidymodels` workflow.

```
1 library(tidymodels)
2
3 set.seed(123)
4 data_split <- initial_split(iris, prop = 0.8)
5 train <- training(data_split)
6 test  <- testing(data_split)
7
8 # Recipe: preprocessing
9 rec <- recipe(Sepal.Length ~ ., data = train)
10
11 # Model specification: linear regression
12 lm_spec <- linear_reg() |> set_engine("lm")
13
14 # Workflow
15 wf <- workflow() |> add_recipe(rec) |> add_model(lm_spec)
16
17 # Fit model
18 fit <- fit(wf, data = train)
```

 Try to run the code in your Script Editor.

The `fit` object

The `fit` object contains the results of the fitted linear regression model:

- **Model call:** a record of the function call used to construct the model.
- **Coefficients:** estimated regression parameters for each predictor variable.
- **Intercept:** the baseline value of the outcome when all predictors are equal to zero.
- **Model summary:** if the underlying model is extracted with `extract_fit_engine()`, additional diagnostics are available.

```
1 fit
```

 Try to run the code in your Script Editor.

Resampling with rsample

Resampling or cross-validation enables to **assess predictive performance**.

- It creates **5 different training/validation sets** without discarding any data.
- This allows you to evaluate model's performance across multiple splits instead of relying on just one train/test split.
- It's especially useful for **robust performance estimation** and reducing bias in model evaluation

```
1 # Create 5-fold cross-validation splits
2 set.seed(123)
3 cv_folds <- vfold_cv(iris, v = 5)
4
5 cv_folds
```

 Try to run the code in your Script Editor.

Hyperparameters with tune

Hyperparameters can be tuned for a random forest using **grids**:

- **rf_tune** defines a random forest model where **mtry** (predictors per split) and **min_n** (minimum node size) will be tuned.
- **rf_grid** creates a grid of candidate values for those parameters.
- **tune_grid()** tests each combination using 5-fold cross-validation and records performance metrics.

```
1 # Define random forest model with hyperparameters to tune
2 rf_tune <- rand_forest(mtry = tune(), trees = 500, min_n = tune()) |>
3   set_engine("ranger") |>
4   set_mode("regression")
5
6 # Define grid of candidate values
7 rf_grid <- grid_regular(mtry(range = c(1,4)), min_n(), levels = 3)
8
9 # Run grid search with cross-validation
10 rf_tuned <- tune_grid(
11   workflow() |> add_recipe(rf_recipe) |> add_model(rf_tune),
12   resamples = cv_folds,
```

```
13     grid = rf_grid
14 )
```

 Try to run the code in your Script Editor.

Take home

- AI/ML can **complement** traditional sampling
- K-means identifies **clusters for stratification**
- Random forest helps **prioritize or weight units**
- R/tidymodels provides **reproducible workflow**
- **Visualization is key** for interpretation

Resources

- James, G., Witten, D., Hastie, T., Tibshirani, R. (2021). *An Introduction to Statistical Learning with Applications in R*. Springer. <https://www.statlearning.com>
- Kuhn, M., & Wickham, H. (2023). *tidymodels: Unified Interface for Modeling in R*. <https://www.tidymodels.org>
- R Core Team. (2025). *R: A Language and Environment for Statistical Computing*. <https://www.R-project.org>
- Grolemund, G., Wickham, H. (2017). *R for Data Science*. <https://r4ds.had.co.nz>

Exercise

Please download the R script with exercises from **GitHub** and try to complete it.